

Routing3D 상세 개발 내역서

플랜트 배관 3D 직교 자동 라우팅 엔진 · 단계별 전체 프로세스 / 핵심 알고리즘 / 주요 함수
/ 변수 / 자료구조

Phase 1 (Python 레퍼런스) → Phase 2 (인터페이스 동결) → Phase 3 (C++ 엔진 + C ABI)
→ C# HelixToolkit 뷰어 → 패턴학습(L1~L4) → 탐색최적화 → DDW_AI_DB 전환

단위: mm · 기본 셀 50mm · DB: DDW_AI_DB · 빌드: C++20 / .NET 9 (x64)

0. 문서 개요

본 내역서는 Routing3D 엔진을 처음부터 현재까지 개발하며 거친 모든 단계의 구현 세부를 한곳에 모은 기술 레퍼런스다. 각 단계는 '전체 프로세스 → 핵심 알고리즘 → 주요 함수(시그니처) → 주요 변수/파라미터 → 자료구조' 순으로 기술한다. 프로젝트의 시간순·요약 현황은 저장소 CLAUDE.md 가, 동결 명세는 docs/spec/*.md 가, 코드 자체가 최종 정답이며 본 문서는 그 지도(map)에 해당한다.

0.1 목차

- 1. 프로젝트 개요 — 목적 / 3-tier 아키텍처 / 환경 / 핵심 불변식(계약)
- 2. Phase 1 — Python 레퍼런스: 점유맵·직교 A*·비용함수·다중배관·scene.txt I/O
- 3. Phase 2 — 인터페이스 동결: 알고리즘/포맷/골든셋/성능목표 명세
- 4. Phase 3 — C++ 엔진: 기하·점유 3 백엔드·박스인덱스·비용·A*·다중배관·계층 corridor
- 5. C ABI DLL — R3dStatus/POD 구조체/진입점/인터럽 안전규칙
- 6. C#↔C++ 인터럽 + HelixToolkit 뷰어 — P/Invoke·SafeHandle·MVVM·3D 렌더·피킹
- 7. 기존설계 패턴 학습(pgvector) — L1 스텝추출 / L2 투영·회랑 / L3 랙·검색증강 / L4 번들탐지
- 8. 탐색 최적화 — ImplicitOccupancy·동적 가중 A*·계층 corridor 게이트·병렬화 기각
- 9. DDW_AI_DB 전환 + 뷰어 표시보강
- 10. 테스트·교차검증·결론

0.2 표기 규약

- 모든 좌표·치수는 밀리미터(mm). 셀(cell)은 정육면체, 기본 한 변 50mm.
- 셀 인덱스 = (i, j, k) 정수, 월드 좌표 = (x, y, z) 실수(mm).
- 함수 시그니처는 실제 소스 기준(Python/C++/C#). 표의 '역할'은 한 줄 요약.
- 불변식(계약)은 C1·M1·M2·O1·F2~F4·A2/W1 처럼 코드 명세(docs/spec)와 동일 기호 사용.

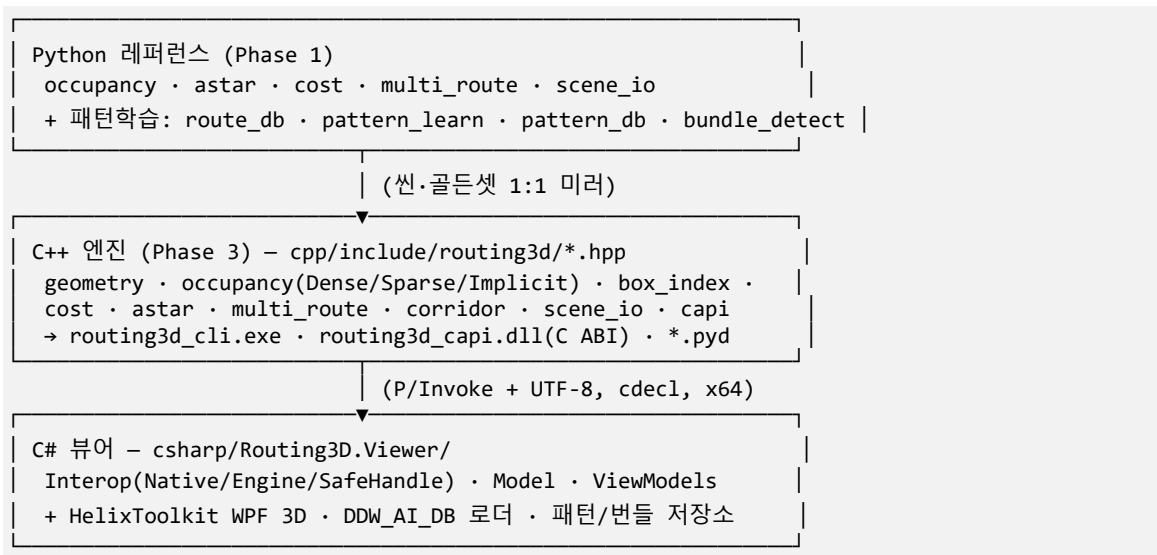
1. 프로젝트 개요

1.1 목적

Routing3D 는 플랜트 BIM(PostgreSQL DDW_AI_DB)의 실제 장애물·장비·종단객체를 입력으로 받아, 배관 종단점(PoC) 사이를 충돌 없이 잇는 3D 직교(맨해튼) 경로를 자동 산출하는 엔진이다. 다수 배관을 서로 충돌 없이 동시에 깔고, 사람이 설계한 기존배관 패턴(스텝·랙·번들)을 학습해 자동 경로에 반영한다.

1.2 3-tier 개발 전략 · 아키텍처

알고리즘을 Python 으로 먼저 검증(Phase 1)하고, 인터페이스·포맷·골든셋을 동결(Phase 2)한 뒤, 동일 동작을 C++ 로 1:1 포팅(Phase 3)한다. C++ 엔진은 C ABI DLL 로 노출되어 C# WPF 뷰어가 P/Invoke 로 호출한다. 단계마다 Python 레퍼런스와 교차검증(바이트/지표 일치)하여 회귀를 막는다.



1.3 개발 환경 · 빌드

항목	내용
단위/셀	전 좌표 mm · 기본 셀 50mm (RouteParams.cell_mm)
Python	루트 .venv · `pip install -e python_experiments[viz]` · `python -m routing3d_py.<mod>`
C++	MSVC VS2022 + CMake · C++20 · /utf-8 필수(한글 주석) · x64 고정

C#	.NET 9 · net9.0-windows · x64 고정(네이티브 DLL 비트 일치) · HelixToolkit.Wpf 2.24.0 · Npgsql
DB	localhost:5432 / DDW_AI_DB / postgres / dinno (env PGHOST 등 우선, 소스에 비밀번호 금지)
확장	pgvector · cube · postgis 설치

1.4 핵심 불변식(계약) — 전 단계 공통

기호	불변식	의미
O1	좌표 일관성	셀 중심 = origin + (cell+0.5)×cell_mm. 모든 점유 백엔드 동일 결과.
A2/W1	A* 결정성	(f, 삽입순서 counter) tie-break + 고정 이웃순서 → 동일 입력→동일 경로/확장수.
M1	다중배관 충돌 0	성공 경로들은 쌍별로 셀을 공유하지 않는다.
M2	원본 점유맵 불변	라우팅은 occ.copy() 사본에서만 수행.
C1	admissibility	비용은 가산 패널티(≥ 0)만 → 휴리스틱 일관성·최적성 보존.
F2	scene.txt 무손실	write→read→write 바이트 동일.
F3	None vs "" 구분	optional<string> None=`\N` (미설정) vs "" (빈문자열) 보존.
F4	repr 표기	format_repr_double = Python repr(float) 최단 표기.

2. Phase 1 — Python 알고리즘 레퍼런스

다섯 모듈로 구성: occupancy(점유맵) → astar(직교 A*) → cost(비용함수) → multi_route(순차 라우팅) → scene_io(텍스트 I/O). 모두 단위 mm, 명확한 계약. 이후 C++ 엔진이 이 동작을 1:1 재현(골든셋 일치)한다. (pytest 200+ 통과)

2.1 occupancy.py — 점유맵

전체 프로세스: 공간을 cell_mm 정육면체 격자로 표현하고 각 셀의 점유 여부를 관리한다. 세 저장 백엔드(Dense/Sparse/BitPacked) 뒤에 통일 질의 인터페이스를 둔다.

- 핵심 알고리즘 — 좌표변환(O1): 셀중심 = $\text{origin} + (\text{cell} + 0.5) \times \text{cell_mm}$. 범위 밖은 항상 점유(경계=벽).
- 복셀화: AABB(lo,hi)를 셀범위로 변환 후 [0,shape) 클리핑.
- 팽창(inflate): 장애물 주변 radius 셀까지 반복 이진 팽창(6/26 연결성).
- 백엔드: Dense(Numpy bool, O(1)) · Sparse(점유셀 set) · BitPacked(Z 8 셀/바이트, 1/8 메모리).

주요 함수	역할
in_bounds(cell)->bool / is_blocked(cell)->bool	범위 내 여부 / 점유·격자밖 여부
to_world(cell)->Vec3 / to_cell(world)->Cell	셀↔월드 좌표 변환
add_box(box)->int	AABB 복셀화, 신규 점유 셀 수 반환
inflate(radius, connectivity)->OccupancyMap	장애물 팽창된 새 맵
copy()->OccupancyMap	독립 사본(M2 보장)
from_world_bounds(lo,hi,cell_mm)	월드 범위 커버하는 빈 맵 생성
주요 변수	타입 · 의미
shape	(nx,ny,nz) 각 축 셀 개수
origin	ndarray[f64] 격자 원점(mm)
cell_mm	float 셀 한 변(기본 50.0)
grid / blocked / packed	Dense bool 배열 / Sparse set / BitPacked uint8 배열

2.2 astar.py — 직교 A*

전체 프로세스: 점유맵 위에서 start→goal 최단 경로를 6 방향($\pm X, \pm Y, \pm Z$) 직교 이동만으로 찾는다.

- 상태: 균일은 셀(i,j,k) / 가중은 (셀, 진입방향). 휴리스틱:
맨해튼거리×cell_mm(admissible).
- 우선순위 큐: (f, counter, cell) — f=g+h, 동률은 삽입순서 counter (결정성 A2/W1).
- 종료: goal 도달 시 경로 복원, open 소진 시 실패. expanded_nodes = 탐색 비용 지표.

주요 함수	역할
astar(occ,start,goal,...)->AStarResult	균일 비용 A*
astar_weighted(occ,start,goal,params,...,corridor)->AStarResult	비용함수 A*(회전·클리어런스·회랑)
manhattan(a,b)->int / count_turns(path)->int	맨해튼 거리 / 방향전환 수
AStarResult 필드	의미
success / path	성공여부 / 경로 [start..goal]
length_mm / cost_mm	기하 길이 / 패널티 포함 비용
turns / expanded_nodes	회전 수 / 확장 노드 수
visited / elapsed_ms	확장 셀(옵션) / 소요시간

2.3 cost.py — 비용함수

전체 프로세스: 기본 이동비용(cell_mm) 위에 가산 패널티 3 종을 더한다 — 회전(엘보 최소화), 클리어런스(벽 회피), 단(層) 분리. 모든 항 ≥ 0 이라 admissibility(C1) 보존.

- move_cost = cell_mm + (w_turn if 방향바뀔) + cell_penalty(to_cell).
- cell_penalty = w_clear×max(0, clearance_radius – 장애물거리) + w_tier[k] + (회랑밖이면 w_corridor).
- clearance_map: 장애물 다중소스 BFS 거리변환(상한 R) — '가장 가까운 장애물까지 셀 거리'.

RouteParams 필드	타입 · 의미 (기본값)
cell_mm	float 셀 기본 비용(50.0)
w_turn	float 회전 1 회 가산(500=셀 10 칸)
w_clear / clearance_radius	float 클리어런스 계수(10.0) / int 적용 반경(2)
clearance_connectivity	int 거리 측정 이웃 6/26 (6)
w_tier	dict[int,float] z 셀→가산 mm ({})
w_corridor / rack_levels	float 회랑밖 가산(0.0) / 선호 단 z 셀({})
w_heur / w_heur_near	float 휴리스틱 가중(1.0) / 동적 목표근처값(0.0) — Phase 3 가중 A*

2.4 multi_route.py — 다중 배관 순차 라우팅

전체 프로세스: 여러 배관을 한 개씩 차례로 라우팅하되, 이미 깔린 배관을 다음 배관의 장애물로 추가 → 배관 간 충돌 0(M1). 원본 점유맵은 불변(M2).

- 우선순위: longest(긴 것 먼저, 기본) · shortest · utility(그룹 ↑, 거리 ↓) · original.
- 절차: order_tasks → occ.copy() → 작업마다 snap_to_free→astar_weighted→성공시 mark_pipe(경로+radius 팽창).
- 회랑(corridor): 깔린 배관 곁 셀 집합 → w_corridor 페널티 면제 → 배관 번들링 유도.
- rip-up & reroute(Phase 3.8): 막힌 배관이 생기면 기존 일부를 걷어내고(rip-up) 재라우팅.

주요 함수	역할
order_tasks(occ, tasks, priority)	우선순위 규칙 정렬
route_sequential(occ, tasks, params, priority, pipe_radius, snap_to_free, ..)	순차 라우팅
route_ripup(occ, tasks, params, ..., max_rounds, max_ripup)	rip-up & reroute
_mark_pipe / _add_corridor / _snap	경로 점유표시 / 회랑추가 / 빈셀 스냅
자료구조	의미
RouteTask{start_mm, end_mm, utility, utility_group}	라우팅 작업(중단점+메타)
PipeResult{task, result, order_index}	배관 1 개 결과
MultiRouteResult{pipes, occupancy, priority}	다중 결과 묶음

2.5 scene_io.py — scene.txt 입출력

전체 프로세스: 입력(격자/파라미터/장애물/작업)과 결과(경로/방문/지표)를 scene.txt 텍스트로 직렬화/역직렬화. Phase 2 명세화·Phase 3 C++ 엔진의 입출력 규약.

- 무손실 왕복(F2): write→read→write 바이트 동일. 실수는 repr(float) 최단표기(F4).
- None vs ""(F3): None='\\N'(PostgreSQL COPY 관례), ""=빈 필드.
- 구조(TAB 구분 섹션): [grid][params][obstacles][tasks][results→[result][path][visited]].

주요 함수	역할
dumps_scene(doc)->str / loads_scene(text)->SceneDoc	문자열 직렬화/역직렬화

write_scene / read_scene	파일 입출력(UTF-8, LF)
occupancy_from_doc(doc, backend)->occupancyMap	grid+obstacles→점유맵 복원

3. Phase 2 — 인터페이스 동결

Phase 1 에서 검증한 동작을 명세로 고정해 Phase 3 C++ 구현이 모호함 없이 따르게 한다.

산출물은 docs/spec/*.md 5 종 + freeze_signoff.

명세 문서	내용
algorithm_spec.md	A* 결정성·비용함수·다중배관·불변식(O1/A2/W1/M1/M2/C1) 동결
scene_format_spec.md	scene.txt v1 규격(섹션·컬럼·₩N·repr) 동결(F2~F4)
regression_set.md	골든 01/02/03 입력+기대지표(정확일치 vs 근사/상한) 동결
performance_targets.md	8,000m 도메인 · 단일 <1 초 · 전체(수백) <1 분 · 메모리 <32GB
freeze_signoff.md	동결 합의 — Phase 3 합격 기준 = 이 지표 재현

4. Phase 3 — C++ 엔진 (헤더 전용 템플릿 + C ABI)

Python 레퍼런스를 C++20 으로 1:1 포팅하고 고성능 확장을 더한다. 점유 백엔드 무관 (컴파일타임 다형성=템플릿)으로 cost/astar/multi_route 가 Dense/Sparse/Implicit 자동 호환. 골든 01/02/03 에서 expanded_nodes 까지 Python 과 정확 일치.

4.1 단계별 구현(Step 3.1~3.12)

Step	내용	검증
3.1~3.4	geometry/occupancy/cost/astar — 헤더 전용 템플릿	골든 01/02 expanded_nodes Python 일치
3.5	multi_route — route_sequential/order_tasks/mark_pipe/snap	골든 03: 5/5-충돌 0-28050mm
3.6	OpenVDB 백엔드 + 계층 corridor(astar_hashed)	8,000m³ 로컬배관 ~75ms
3.7	FCL 정밀 충돌(fcl_scene) sub-voxel 캡슐	틈 200mm 가는/굵은 파이프 구별
3.8	rip-up & reroute(무손실 결정적)	합성 1/2→2/2 · project6 +3
3.9	scene.txt I/O(Python 바이트 동일)	F2 무손실 왕복
3.10	pybind11 바인딩 → routing3d_cpp.pyd	골든 01/02/03 + 왕복 일치
3.11	sparse 확장 ImplicitOccupancy + 64 비트 키 + 온디맨드 클리어런스	Dense==Implicit 전수 일치 · 10mm 크래시 0
3.12	회귀 리포트(표준 벤치 자동 측정)	골든 3/3 PASS

4.2 geometry.hpp — 기하/격자 기본 타입

- Cell(int i,j,k) · Vec3(double x,y,z) · AABB(Vec3 lo,hi) · NEIGHBORS_6 · CellRange(lo,hi).
- grid_in_bounds / grid_cell_to_world / grid_world_to_cell / grid_box_range / manhattan.
- Python 과 1:1. 좌표 규약 O1 동일.

4.3 occupancy.hpp — 점유 3 백엔드

불변식 O1: 같은 입력에 모든 백엔드 is_blocked 결과 동일(ctest implicit 로 Dense==Implicit 전수 검증).

백엔드	저장 · 적합 상황
DenseOccupancy	vector<uint8_t> 1B/셀 · 작은 ROI · 질의 O(1) · 골든 바이트 불변
SparseOccupancy	unordered_set<uint64_t> 64 비트 패키징 · 희박/거대 격자 · corridor

ImplicitOccupancy	SpatialBoxIndex + marked set · 복셀화 폐기 → 메모리 O(장애물+칼린셀), 셀 크기 무관
-------------------	---

- 공통 인터페이스:
in_bounds/is_blocked/to_world/to_cell/block_cell/add_box/copy/count_blocked/shape/origin/cell_mm.
- A* 키: lin(cell)->정수 / unlin(키)->cell. Dense=int, Sparse/Implicit=long long(오버플로 없음).
- ImplicitOccupancy.clearance_cells(cell, max_radius): 온디맨드 클리어런스(전역 거리변환 폐기).

4.4 box_index.hpp — 공간 박스 인덱스 (Step 3.11)

장애물 AABB 를 유니폼 그리드 해시(broadphase)로 색인. ImplicitOccupancy 가 사용 → 메모리 O(장애물 수).

주요 함수	역할
SpatialBoxIndex(bucket_mm) / add(lo,hi)	버킷 크기 생성 / 장애물 AABB 추가
overlaps(qlo,qhi)->bool	질의 박스와 겹치는 장애물 존재 여부(점유 판정)
nearest_dist(p,max_dist)->double	점→최근접 장애물 표면 거리(온디맨드 클리어런스 상한)

4.5 cost.hpp / astar.hpp — 비용·탐색 (템플릿)

- HasClearanceQuery concept: 백엔드가 온디맨드 클리어런스 제공 시 전역 거리배열 미생성(메모리 $\propto$ 탐색셀).
- astar_weighted 상태키 = lin(셀)×7 + (dir+1) [dir∈-1..5] → 64 비트 안전(S2).
- 동적 가중 A*: $w_{eff} = w_{heur_near} + (w_{heur} - w_{heur_near}) \cdot h / h_{start}$ (먼곳 빠름·목표근처 정확).
- on_progress 콜백 + in_corridor 술어(기본 AllowAll=골든 불변, escalation 게이트용).

주요 함수(템플릿)	역할
clearance_map<Occ>(occ,max_radius,connectivity)	거리변환 배열(Dense/Sparse)
CostModel<Occ>(occ,params,corridor)	cell_penalty / move_cost / heuristic
astar<Occ>(occ,start,goal,...)	균일 A*
astar_weighted<Occ,InCorridor>(occ,start,goal,params,...,on_progress)	가중 A*(동적·진행콜백·회랑술)

4.6 multi_route.hpp / corridor.hpp

- route_sequential<Occ>(occ,tasks,params,priority,pipe_radius,snap_to_free,...,corridor_radius): 충돌 0(M1)·원본불변(M2).
- order_indices/order_tasks · snap_to_free_cell · mark_pipe · add_corridor_cells(w_corridor>0 시 자기 번들링).
- corridor: pack20/unpack20(축당 20 비트, 8,000m/50mm=160k 커버) · astar_hashed(closed 해시 → 거대격자) · route_corridor(coarse 가이드→fine 튜브).

자료구조	정의
RouteTask	{Vec3 start_mm,end_mm; optional<string> utility,utility_group,start_name,end_name,end_instance_guid}
MultiRouteResult<Occ>	{vector<PipeResult> pipes; Occ occupancy; string priority}
CorridorRoute	{AStarResult fine; vector<Cell> coarse_path; bool coarse_success; long long corridor_cells}

5. C ABI DLL — routing3d_capi.dll

C++ 엔진을 extern "C" C ABI 로 노출 → C#(P/Invoke)·Python(ctypes) 등 어떤 호스트도 인프로세스 호출 가능. 외부 의존성 0. ABI 안전: 예외는 경계를 넘지 않고 R3dStatus 로 변환, STL 미노출(불투명 핸들 + POD blittable 구조체), cdecl, x64, 문자열 UTF-8.

R3dStatus	의미
R3D_OK=0	성공
R3D_ERR_ARG=1 / R3D_ERR_PARSE=2	잘못된 인자 / scene.txt 파싱 실패
R3D_ERR_RUNTIME=3 / R3D_ERR_RANGE=4	실행 중 예외 / 인덱스·범위 오류
POD 구조체	필드
R3dGrid	double cell_mm, ox,oy,oz; int32 nx,ny,nz
R3dParams	double cell_mm,w_turn,w_clear,w_corridor,w_heur,w_heur_near; int32 clearance_radius,clearance_connectivity,corridor_radius,rack_level_count; int32 rack_levels[8]
R3dResult	int32 success; double length_mm,cost_mm; int32 turns; int64 expanded_nodes; double elapsed_ms; int32 path_len,visited_len
주요 진입점	역할
r3d_create / r3d_destroy	엔진 생성/소멸(불투명 핸들)
r3d_load_scene_text / r3d_dump_scene_text	scene.txt 적재 / 현재 상태 직렬화
r3d_set_grid / r3d_set_params	격자·파라미터 설정
r3d_add_obstacle / r3d_add_passthrough / r3d_add_task	장애물/통과객체/작업 추가(task→인덱스)
r3d_set_task_endpoints	작업 종단점 갱신(인터랙티브 재라우팅)
r3d_route_multi / r3d_route_multi_progress	순차 라우팅(충돌 0) / +진행 콜백
r3d_route_task	단일 작업 라우팅(원본 장애물)
r3d_route_ripup	rip-up & reroute(max_rounds,max_ripup)
r3d_route_corridor / r3d_route_corridor_multi	계층 corridor(독립/순차)
r3d_set_corridor_cells	학습된 회랑 셀 주입(L2b, w_corridor>0 시)
r3d_get_result / r3d_copy_path / r3d_copy_visited	결과 조회 / 경로·방문 셀 복사(2 단계 버퍼)
r3d_copy_blocked / r3d_copy_passthrough	점유·통과 셀 복사(가시화)
r3d_free_string	콜리 할당 문자열 해제

진행 콜백 R3dProgressFn(cdecl): phase 0=탐색 진행(progress01), phase 1=배관 완료(success/길이/path_ijk). path_ijk 는 콜백 동안만 유효 → 즉시 복사.

6. C#↔C++ 인터롭 + HelixToolkit WPF 뷰어

C# WPF 뷰어가 routing3d_capi.dll 을 P/Invoke 로 호출해 라우팅하고, HelixToolkit 으로 3D 렌더링한다. MVVM(ObservableObject/RelayCommand), SafeHandle 수명관리, 비동기 DB 로드.

6.1 인터롭 3 계층(Interop/)

파일	역할 · 핵심
Native.cs	P/Invoke 선언(cdecl). 문자열=UTF8 바이트(널종료). R3dGrid/Params/Result blittable. r3d_* 진입점.
R3dEngineHandle.cs	SafeHandleZeroOrMinusOneIsInvalid — GC/using 시 r3d_destroy 정확히 1 회(이중해제·누수 0).
Engine.cs	예외 기반 OOP 래퍼. Check(status)→0 아니면 throw. 콜백 중 path Marshal.Copy 즉시 복사 + GC.KeepAlive.

- PathCell(int I,J,K) record · RouteResult{success,length,turns,path[],visited[]}
- Engine 메서드:
LoadSceneText/SetGrid/SetParams/AddObstacle/AddTask/SetTaskEndpoints/RouteMulti/RouteMultiProgress/RouteTask/RouteCorridor(Multi)/SetCorridorCells/GetResult/CopyBlocked/DumpSceneText.

6.2 모델(Model/)

클래스	역할
SceneData	격자/장애물/작업/장비/덕트/공간/기존배관/부속 컨테이너(+RawText)
GridMeta / ObstacleBox / TaskInfo	격자메타 / 장애물 AABB+OST·통과여부 / 작업 중단점+유틸
EquipmentBox / DuctLateral / SpaceArea	장비 / 덕트·레터럴 / 공간영역(층) AABB
ExistingPipe / PipeFitting	기존배관 폴리라인+GUID+관경+PoC / 배관자재(ELBOW/TEE/VALVE...)
SceneTextParser	scene.txt v1 파서([grid][obstacles][tasks], WN=null)
UtilityColors	유틸 라벨 Ordinal 정렬 → 24 색 팔레트 순환(결정적)
CollisionFinder	여러 경로에서 ≥2 공유 셀 탐지(단일 재라우팅 충돌 검출)
ObstacleDbLoader	DDW_AI_DB 로더(그룹 AABB 스코프, §9)

PatternStore / BundleStore / StubExtractor	스텝패턴(L2a) / 번들(L4) / 스텝추출(L2a 미러)
--	-----------------------------------

6.3 뷰모델(ViewModels/) · 3D 렌더 · 피킹

- SceneViewModel(중앙, 3900+줄): 엔진+렌더+인터랙션 조율. BuildModel 이 HelixToolkit 기하 구성(장애물 박스·경로 TubeVisual3D(유틸색)·충돌 큐브·기준배관/공간/장비/덕트 선택 렌더).
- TaskRowVM: 작업 1 행(Index/Label/Swatch/종단점 Sx..Gz/Success/LengthMm/Path/Visited/DiameterMm/StartStub/EndStub/TurnCount).
- UtilityFilterVM: 유틸 체크박스(IsVisible 토글→재렌더). ObservableObject: Set<T>(ref,value)·OnChanged. RelayCommand: ICommand.
- 3D 클릭 피킹: FindNearestPoint(화면)→SelectObjectAt(월드) AABB 포함검사·부피최소(가장 구체적) 우선; 배관은 중심선 거리.
- PickMode(None/Start/End): 3D 클릭으로 종단점 지정→즉시 재라우팅(인터랙티브).
- 라우팅 모드: Shortest(순수 A*) / GroupPattern(코너 waypoint 경유) / FollowExisting(기준배관 복제+국소수리).

6.4 인터럽 단계(P0~P3j)

단계	내용
P0/P1	C ABI DLL + WPF-HelixToolkit 뷰어(route_multi-3D 렌더)
P2/P3a	인터랙티브 재라우팅(종단점 편집) / 충돌 시각화·표시 토글·3D 클릭 종단점
P3b/P3c	corridor C ABI(Sparse) / scene.txt CLI---selftest 헤드리스
P3d/P3e	SpaceAI 다크 3 컬럼 UI·검색·유틸 필터 / 3D 레이어(복셀/점유/방문맵)
P3f/P3g	DB 자동 로드 / 워크플로 재설계(창 즉시+비동기 로드·탐색범위 선택·드릴다운)
P3h/P3i	DB 레이어 확장(장비·덕트·공간·PoC 명) / 탐색 시각화(A* 애니메이션·구간 리스트)
P3j	기존설계 패턴 학습(\$7) — 학습면 투영·접근불가 스냅·기존설계 회랑

6.5 데이터 흐름

[DB / scene.txt]
→ ObstacleDbLoader.LoadScene / SceneTextParser.Parse

- SceneData(격자/장애물/작업/...)
- SceneViewModel: Engine.SetGrid/SetParams/AddObstacle/AddTask
- Engine.RouteMulti() —P/Invoke—> routing3d_capi.dll (C++ astar_weighted)
- RouteResult → TaskRowVM.Path/Visited/Success/LengthMm
- BuildModel(): 장애물 박스 + 경로 TubeVisual3D + 충돌 큐브
- HelixViewport3D 렌더

7. 기존설계 패턴 학습 (pgvector, L1~L4)

사람이 설계한 기존배관(TB_ROUTE_PATH)의 형상을 학습해 자동 라우팅에 활용한다.
스텝(장비 출발·덕트 진입 형상)·랙(공용 수평 높이)·번들(평행 다발)을 추출·저장·추론한다.
엔진 골든 불변(회랑/랙은 w_corridor>0 또는 rack_levels 주입 시에만 동작).

7.1 단계 개요

단계	내용	효과(project6 c100 기준)
L1	pgvector 학습소 + 스텝 추출 파이프라인	405 표본/38 키 · 도메인규칙(EQUIP=-z·DUCT=+z) 입증
L1' 엘보	스텝을 '수직+첫 엘보'까지로(런압축·지터흡수·엘보탐지)	엘보 방향이 dir2 에 인코딩
L2a	학습면 PoC 투영(C#)	기하 187 · 38s→23s(헛탐색 0)
전처리	접근불가(파문힌) PoC→최근접 자유셀 스냅	187→199 · 23s→6.3s
L2b	기존설계 회랑 소프트바이어스(옴트인)	199(동일성공)·설계추종(totalLen ↓)
L3a 랙	유틸그룹 수평런 z- 높이→rack_levels(랙 번들)	199→200 · 랙집중 17%→21%
L3b 검색증강	다중면 DUCT 키 ANN 면 분기 + LOO 자기검증 게이트	무분별 ANN 해로움→게이트로 NFW 1 키만 채택(회귀 0)
스텝 라우팅	매칭 스텝을 고정 구간으로, A*는 스텝끝~끝만 탐색	199→206(랙집중 17%→37%)
L4 번들	평행 다발 탐지→공용 트렁크 z-pitch 활용	70 프로젝트·353 그룹·275 키 적재

7.2 route_db.py — 기존배관·장비·덕트 로더 (L1)

- list_groups/resolve_group: TB_SPACE_GROUP_INFO 그룹(툴)=프로젝트(1-based).
- load_equipment/load_ducts/load_existing_pipes(xy_bbox): 그룹 AABB XY 교차 스코프.
- load_existing_pipes: SEGMENT_DETAIL 3-join 폴리라인(points≥2). parse_pipe_size_mm:
'40A'→40,'1/2B'→12.7.
- SCOPE_MARGIN_MM=500.0. 자료구조
GroupInfo/EquipmentBox/DuctLateral/ExistingPipe(owner_name=번들키).

7.3 pattern_learn.py — 스텝 추출/특징벡터 (L1~L3a)

핵심 알고리즘 _walk_stub: PoC 폴리라인에서 '수직배관 + 첫 엘보(수직→수평 전환)'까지를 스텝으로 절단.

- ① _dir_runs(seg): 폴리라인→방향 런[(축 0..5,누적길이)] 압축(연속 동일방향 병합).
- ② _merge_short_runs(runs): STUB_MIN_DIR_RUN_MM(250) 미만 런=지터→인접 런에 흡수(엘보 오인 방지).
- ③ 첫 런 축=수직축. 수직축과 다른 첫 런=엘보. 엘보 포함, 엘보 후 수평 리드인 STUB_LEADIN_MM(800)만.
- build_feature_vector(24 차원): [face 6][1 차방향 6][2 차방향 6][앵커내 상대좌표 3][진행 단위벡터 3].
- learn Rack_levels(L3a): 유틸그룹 수평런(min_run 800mm) 중점 z 를 bin(100mm) 버킷 누적→주 랙 높이.

상수	값 · 의미
STUB_MAX_MM / STUB_MAX_BENDS	4000mm 스텝 최대길이 / 3 최대 꺾임
STUB_MIN_DIR_RUN_MM / STUB_LEADIN_MM	250mm 지터 임계 / 800mm 엘보후 리드인
ANCHOR_MAX_MM	3000mm 앵커(장비/덕트) 매칭 허용 반경
RACK_HORIZ_MIN_MM / RACK_BIN_MM	800mm 최소 수평런 / 100mm z 버킷
FEAT_DIM	24 특징벡터 차원

7.4 pattern_db.py — pgvector 저장소/추론 (L2~L3b)

- apply_schema/clear_source/insert_samples: route_stub_pattern 적재(vector 는 vec_literal '::vector' 캐스트).
- nearest_stubs: 범주(kind/group/util) WHERE pre-filter → feat L2 거리 ANN K 개(범주는 절대 제약).
- suggest_stub→StubSuggestion: K-NN 거리가중($1/(dist+\epsilon)$) 투표 → face-face_conf-rise-offset.
- LOO 자기검증 게이트(L3b): leave-one-out 으로 ANN 이 집계 다수결을 +10pp 이기는 키만 ANN 채택(무분별 ANN 은 UPW_S/HOT DI_S 에서 rel/dir 이 면과 반상관 → 회귀 199→192 방지, NFW 1 키만 적용).

7.5 bundle_detect.py — 번들 탐지 (L4)

핵심 알고리즘 3 단계: 특징추출 → 복합유사도 → 공간 동시진행 검출(v3).

- Phase1 특징: arrow_code(R 수직/H 수평/D 경사) · count_ortho_bends · resample(24 점) 방향벡터 · extent · trunk_axis.
- Phase2 복합유사도 = 형태 30%(1-Levenshtein) + 방향 30%(코사인, 양방향 max) + 길이 20% + 규모 20%.
- Phase3 동시진행: _extract_runs(레인유지 ±LANE_TOL_MM=10) → (owner,util) pre-filter → 축별 평행다발(공유좌표 클러스터·진행겹침 Union-Find·등간격 분할 PITCH_GAP_FACTOR=2.5) → 코너병합 → trunk z-pitch.
- aggregate_templates: (owner,util) 키별 수평 트렁크 고도(trunk_axis<2)·pitch 중앙값·최빈 arrow_code.

상수(v3)	값 · 의미
LANE_TOL_MM / MIN_RUN_MM / MIN_OVERLAP_MM	10mm 레인 유지 / 800mm 최소런 / 300mm 최소겹침
PITCH_GAP_FACTOR / MIN_RACK_MEMBERS	2.5 등간격 분할 / 2 최소 멤버
W_SHAPE/W_DIR/W_LEN/W_SCALE	0.30/0.30/0.20/0.20 유사도 가중 · RESAMPLE_N=24

7.6 DB 스키마 (pgvector)

테이블/뷰	핵심 컬럼 · 역할
route_stub_pattern	feat vector(24)·dir_unit vector(3)·face_dir_seq·rise_mm·offset_mm + HNSW(l2/cosine)
route_stub_template(뷰)	(kind,group,util) 키별 mode(face/dir_seq)·percentile(rise/offset) = 폴백 1 순위
route_bundle_group	owner_name·utility·trunk_z·pitch_mm·trunk_axis·member_guids[] · n_members
route_bundle_template(뷰)	(source_file,owner,util) 키별 수평 trunk_zs[]·pitch 중앙값 = 신규설계 활용

DDW_AI_DB에서는 공식 학습

테이블(TB_ROUTE_SEGMENT_TEMPLATE·TB_ROUTE_DESIGN_GROUP)을 사용하며, 위 route_stub_pattern/route_bundle_group 은 자체 학습 레퍼런스 스키마다(§9).

8. 탐색 최적화 (정밀 셀 · 거대 격자)

미세격자(10~25mm)·초대형 도메인(8,000m)에서 메모리·탐색량을 근본 축소한 일련의 최적화. 모두 골든(50mm·w_heur=1) 결과 불변을 유지하도록 게이트를 둔다.

기법	원리 · 효과
ImplicitOccupancy (3.11)	복셀화 폐기·장애물 AABB 를 SpatialBoxIndex 색인 → 메모리 $O(\text{장애물} + \text{칼린셀})$, 셀 크기 무관. 10mm 20.3 억셀 크래시 0.
64 비트 상태키 (S2)	astar_weighted 상태= $\text{lin} \times 7 + (\text{dir} + 1)$ → 거대 격자 오버플로 해소.
온디맨드 클리어런스 (S4)	전역 거리변환 폐기·CostModel HasClearanceQuery 로 셀별 질의 → 메모리 $\propto$ 탐색셀.
스텝 라우팅	매칭 스텝을 고정 구간으로 깔고 A* 는 랙↔랙만 탐색 → 미세격자 탐색량 근본 해소(206/208).
동적(수렴) 가중 A*	$w_{\text{eff}} = w_{\text{heur_near}} + (w_{\text{heur}} - \text{near}) \cdot h / h_{\text{start}}$. 먼곳 2.0(빠름)·목표근처 1.0(정확). 무제한 탐색에서만(거대격자 자동 off). 스텝 ON 206→208(완전).
계층 corridor 게이트	저예산(300k) 직접 A* 먼저 → 초과 '어려운 배관'만 coarse 가이드→fine 튜브 hier. 쉬운 배관은 직접(회귀 0). 최악 10mm 41s→28s.
독립 배관 병렬화(기각)	optimistic 병렬은 순차와 바이트 동일했으나 wall- clock 이득 0~음수(메모리대역 바운드) → 제거. 교훈: 가속=탐색량 축소(스텝·동적가중·hier).

실측(project6 ALL 208 작업): cell=50 0.14s · cell=25 0.5s · cell=10 7.7s(스텝 ON, 208/208
매칭). 옛 '10mm ~110s/배관'은 스텝/Implicit/가중 A* 이전 수치. ctest 11/11 불변.

9. DDW_AI_DB 전환 + 뷰어 표시보강

구 AUTOROUTINGV7 에서 DDW_AI_DB 로 데이터 소스를 완전 교체했다. 프로젝트=툴그룹, 작업·기존배관은 TB_ROUTE_PATH 가 정본, PoC 는 별도 테이블, 좌표는 AABB_*. C# 로더(ObstacleDbLoader)를 전면 재작성했다(Python 레퍼런스는 후순위).

9.1 핵심 테이블

테이블	용도 · 키
TB_SPACE_GROUP_INFO	프로젝트=툴그룹. TAG_GROUP_*·AABB_MIN/MAX_* (공간 스코프 박스)
TB_BIM_OBSTACLE	장애물 AABB · INSTANCE_NAME · OST_TYPE · COLLISION_PASS(0/1 통과플래그) · damper 제외
TB_EQUIPMENTS	장비. MAIN_SUB_TYPE('MainTool') · AABB_*
TB_POCINSTANCES	PoC 인스턴스. INSTANCE_ID(=ROUTE_PATH.SOURCE/TARGET_GUID) · UTILITY
TB_LATERAL_PIPE / TB_DUCT	중단객체(구 TB_DUCT_LATERAL 분리)
TB_ROUTE_PATH(+SEGMENTS/DETAIL)	작업·기존배관 정본. SOURCE/TARGET_POS·OWNER_NAME·BEND_COUNT·TOTAL_LENGTH
TB_ROUTE_DESIGN_GROUP	공식 번들그룹(L4 대체). GROUP_ID·MEMBER_ROUTE_GUIDS[]
TB_ROUTE_SEGMENT_TEMPLATE	공식 스텝 템플릿(L2a). SEGMENT_ROLE(A_EQUIP_STUB/C_DUCT_ENTRY)·START/END_DIR_UNIT·LOCAL_POINTS_JSON

9.2 격자 3 축 그룹 AABB 클램프 (셀 폭증 차단)

- 공유 슬래브(바닥/천장 XY 수백 m·전고 기둥 Z 48m)가 공간필터에 걸려 28 억 셀 폭증 → origin-shape 를 그룹 AABB 박스(3 축)로 클램프(ComputeGrid). 실측 WTNHJ02 136×149×91 ≈ 184 만 셀.
- 장애물·공간영역도 그룹 AABB 로 클리핑(라우팅 불변 — 엔진은 장애물∩격자만 복셀화). 공간영역=와이어프레임.
- 작업 끝점은 XY 로만 스코프되므로 격자 Z 밴드를 작업 끝점까지 확장(그룹 Z 밖 종단의 조용한 실패 방지).

9.3 뷰어 표시보강

- 배관 자재(부속): TB_ROUTE_SEGMENT_DETAIL.TYPE 이 PIPE/POC/BENDING 이 아닌 것(ELBOW/TEE/VALVE/FLANGE...)을 FROM/TO 중점에 타입별 색 cube 로(토글 ShowFittings).

- 3D 클릭 정보 확장: 배관자재·기존배관(중심선 거리)·PoC 마커까지
클릭→ObjectInfoWindow 속성 표시.
- 종단 주의: 종단이 Duct 가 아니라 Damper/Elbow 로 보이는 건 정상(배기 배관이 덕트
부속에 접속).
- 전수 분석 엑셀: _gen_route_analysis.xlsx.py → 전체 7,052 경로 + 종단소유자
교차표(openpyxl).
- PatternStore→TB_ROUTE_SEGMENT_TEMPLATE ·
BundleStore→TB_ROUTE_DESIGN_GROUP 로 공식 학습자산 사용.

10. 테스트 · 교차검증 · 결론

10.1 테스트 체계

계층	명령 · 기대
C++	ctest --test-dir cpp/build -C Release → golden-scene_io-occupancy-corridor-implicit-ripup-attract-capi-vdb-fcl (10/10+)
Python	.venv\python -m pytest python_experiments → 200+ 통과
C# 헤드리스	Routing3D.Viewer.exe --selftest scene.txt out.txt / --dbroute <proj> <cell> ALL <out>

10.2 실데이터 3 자 교차검증 (Python = C++ = C#)

썬	결과
project6_c100(장애물 983·작업 208)	194/208 · 3,400,800mm — 3 자 완전 일치
project6(cell=200)	multi 77 / ripup 80(+3) — rip-up 실데이터 개선
합성 혼잡(9×9 벽+툼 2)	seq 1/2 → ripup 2/2 — C++/Python 동일
DDW WTNHJ02(cell=100)	스텝 ON 151/151 매칭 · 번들 149 키 · 격자 184 만 셀

10.3 결론 · 남은 과제

- 충돌 없는 직교 경로를 안정 생성하며, 길이/굴곡이 사람 설계 대비 합리적(우회비 ↓·더 직선적).
- 패턴학습(스텝·랙·번들)으로 사람 설계 추종(랙 집중도 17%→37%, 스텝 208/208).
- 남은 과제: negotiated-congestion/CBS(혼잡·막힘 잔여 실패), Python 레퍼런스 DDW 재작성, OpenVDB C ABI.

본 내역서의 함수/변수/알고리즘은 작성 시점의 소스 기준이다. 최신 상태는 저장소 CLAUDE.md 와 git 이력, 그리고 각 모듈 소스가 최종 정답이다.